



نظري

6



15



1800 sp

كلية الهندسة المعلوماتية

السنة الثالثة

SQL SERVER PART 2

د.روان قرعوني



RB Informatics; 29/06/2025

محتوى مجاني غير مخصص للبيع التجاري

قواعد معطيات

في المحاضرة السابقة، تعرّفنا على المبادئ الأساسية للتعامل مع SQL Server من منظور عملي، وتعلمنا كيف نترجم بعض مفاهيم الجبر العلائقي إلى أوامر SQL حقيقية يمكن تنفيذها داخل بيئة SQL Server Management Studio (SSMS) في هذه المحاضرة: سنكمل ما بدأناه بالتعمق أكثر في الاستعلامات المتقدمة .

أولاً Aggregate Functions (الدوال التجميعية):

الفكرة العامة:

الدوال التجميعية هي دوال تُستخدم لتطبيق عملية تلخيص أو تجميع على مجموعة من الصفوف داخل جدول، بدلاً من العمل على صف واحد فقط. تُستخدم غالباً مع جملة SELECT، وتعيد قيمة واحدة ناتجة عن معالجة عدة صفوف.

أنواع الدوال التجميعية الأساسية:

الدالة	الوظيفة
COUNT()	تحسب عدد الصفوف
SUM()	تحسب مجموع القيم الرقمية
AVG()	تحسب المتوسط الحسابي
MIN()	تُعيد أصغر قيمة
MAX()	تُعيد أكبر قيمة

أمثلة عملية:

1. عدد الطلاب:

SELECT COUNT(*) FROM Students;

(يُحسب عدد كل الصفوف (حتى وإن كانت تحتوي قيماً NULL)

2. مجموع الرواتب:

SELECT SUM(salary) FROM Employees;

3. متوسط الأعمار:

SELECT AVG(age) FROM Students;

4. أكبر و أصغر قيمة:

SELECT MIN(age), MAX(age) FROM Students;

ملاحظة مهمة جداً:

جميع الدوال التجميعية تتجاهل القيم الفارغة NULL أثناء الحساب ما عدا COUNT(*).

مثال توضيحي:

id	name	age
1	Ali	20
2	Sara	null
3	Noor	22

SELECT COUNT(age) FROM Students; 2 (لا تحسب NULL)

SELECT COUNT(*) FROM Students; 3 (تُحسب كل الصفوف)

SELECT AVG(age) FROM Students; 21 (تتجاهل null)

الفائدة من Aggregate Functions

- تُستخدم في التقارير والإحصاءات.
- ضرورية في عمليات التحليل.
- يمكن ربطها بجملة GROUP BY لتجميع النتائج حسب مجموعات (سنناولها في الفقرة القادمة).

ثانياً GROUP BY + HAVING :

الفكرة العامة:

GROUP BY تستخدم مع الدوال التجميعية لتجميع الصفوف المتشابهة حسب قيمة معينة في عمود واحد أو أكثر، وتنفيذ دالة تجميعية على كل مجموعة على حدة.

بدون GROUP BY تطبق الدالة التجميعية على كل الجدول مرة واحدة فقط.

أما مع GROUP BY يتم تجميع الصفوف أولاً حسب القيمة المطلوبة، ثم تطبيق الدالة على كل مجموعة.

مثال عملي :

■ بفرض لدينا جدول الموظفين التالي:

emp_id	department	salary
1	IT	5000
2	HR	3000
3	IT	5500
4	HR	3200
5	Sales	4000

1. بدون (GROUP BY):

```
SELECT AVG(salary) AS avg_salary
```

```
FROM Employees;
```

2. النتيجة:

avg_salary
4140

■ يحسب متوسط الرواتب لجميع الموظفين بدون تمييز بين الأقسام.

3. مع (GROUP BY):

```
SELECT department, AVG(salary) AS avg_salary
```

```
FROM Employees
```

```
GROUP BY department;
```

department	avg_salary
IT	5250
HR	3100
Sales	4000

تم تجميع الصفوف حسب القسم (department) ، ثم حُسب متوسط الراتب لكل قسم على حدة.

متى نستخدم GROUP BY ؟

- عندما تريد إجراء تجميع وتحليل حسب فئة معينة (مثل القسم، المدينة، الفئة العمرية)
- عند كتابة دالة تجميعية + عمود غير تجميعي في SELECT.

قواعد مهمة (بدونها يظهر خطأ)

كل عمود في SELECT يجب أن يكون:

إما داخل دالة تجميعية (AVG, SUM, ...)

أو مستخدماً في GROUP BY.

يمكن استخدام أكثر من عمود في GROUP BY للفئات المركبة.

لاحظ في المثال السابق تطبيق القاعدة :

department تم استخدامه في group by

salary تم استخدامه داخل الدالة التجميعية

لو تم استخدام عمود ثالث في ال select غير موجود في group by ولا في دالة تجميعية لظهر لدينا خطأ أثناء التنفيذ.

مثال على GROUP BY بأكثر من عمود:

بفرض جدول الموظفين:

emp_id	department	gender	salary
1	IT	M	5000
2	HR	F	3000
3	IT	F	5500
4	HR	F	3200
5	Sales	M	4000
6	HR	M	3100
7	Sales	M	4200

```
SELECT department, gender, COUNT(*) AS count
FROM Employees
GROUP BY department, gender;
```

النتيجة:

department	gender	count
IT	M	1
IT	F	1
HR	F	2
HR	M	1
Sales	M	2

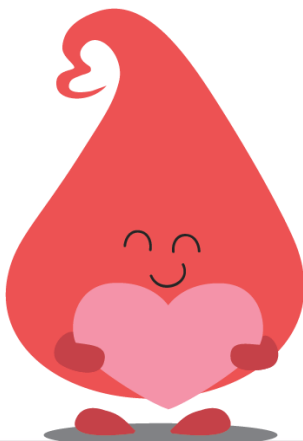
↪ HAVING – تصفية المجموعات بعد التجميع:

الفكرة العامة:

- عندما نستخدم GROUP BY لتجميع البيانات، لا يمكننا استخدام WHERE لتصفية النتائج الناتجة عن الدوال التجميعية (SUM, AVG, COUNT, ...)، لأن WHERE تعمل قبل التجميع.
- هنا تأتي وظيفة HAVING
- تُستخدم لتصفية النتائج بعد تنفيذ GROUP BY، بناءً على قيم الدوال التجميعية.

↪ الفرق بين WHERE و HAVING:

الجملة	تُستخدم قبل/بعد التجميع؟	ماذا تُفعل؟
WHERE	GROUP BY قبل	تُفعل الصفوف قبل التجميع
HAVING	GROUP BY بعد	تُفعل المجموعات الناتجة من التجميع



مثال توضيحي:

بفرض جدول الموظفين :

emp_id	department	salary
1	IT	5000
2	HR	3000
3	IT	5500
4	HR	3200
5	Sales	4000

استعمال بدون HAVING:

```

SELECT department, AVG(salary) AS avg_salary
FROM Employees
GROUP BY department;
```

النتيجة:

department	avg_salary
IT	5250
HR	3100
Sales	4000

استعمال مع HAVING:

```

SELECT department, AVG(salary) AS avg_salary
FROM Employees
GROUP BY department
HAVING AVG(salary) > 4000;
```

النتيجة:

department	avg_salary
IT	5250

فقط القسم الذي متوسط راتبه أكبر من 4000 سيظهر.

```
SELECT department, COUNT(*) AS emp_count
FROM Employees
GROUP BY department
HAVING COUNT(*) >= 2;
```

يعرض الأقسام التي فيها اثنين أو أكثر من الموظفين.

يمكن استخدام WHERE و HAVING معاً:

```
SELECT department, AVG(salary)
FROM Employees
WHERE salary > 2000
GROUP BY department
HAVING AVG(salary) > 4000;
```

هنا WHERE تستبعد الرواتب الصغيرة قبل الحساب، و HAVING يتم فلترتها بعد التجميع.

فائدة أربيسيزية أخطاء شائعة:

الخطأ 1: استخدام دوال تجميعية في WHERE :

المثال الخاطئ:

```
SELECT department, AVG(salary)
FROM Employees
WHERE AVG(salary) > 4000
GROUP BY department;
```



!!!! لا يمكن استخدام AVG أو أي دالة تجميعية داخل WHERE.

التصحيح:

استخدام HAVING للتصفية بعد التجميع:

```
SELECT department, AVG(salary)
```

```
FROM Employees
GROUP BY department
HAVING AVG(salary) > 4000;
```

الخطأ 2: استخدام HAVING بدون GROUP BY بشكل خاطئ :

المثال الخاطئ:

```
SELECT department
FROM Employees
HAVING salary > 3000; ❌
```

!!!! لا معنى لاستخدام HAVING هنا لأنها لا تعمل على الصفوف، بل على مجموعات.

التصحيح:

إذا أردنا الفلتر قبل التجميع، نستخدم WHERE:

```
SELECT department
FROM Employees
WHERE salary > 3000;
```

الخطأ 3: استخدام HAVING بدلاً من WHERE لتصفية الصفوف الفردية:

المثال الخاطئ:

```
SELECT department, COUNT(*)
FROM Employees
GROUP BY department
HAVING salary > 3000; ❌
```

!!!! هنا نحاول فلتر رواتب الأفراد باستخدام HAVING، وهذا لا يعمل.

التصحيح :

استخدام WHERE لتصفية الصفوف قبل التجميع:

```
SELECT department, COUNT(*)
FROM Employees
WHERE salary > 3000
GROUP BY department;
```


ثالثاً Subquery – الاستعلامات الفرعية في SQL :

الفكرة العامة:

الـ Subquery هو استعلام يُكتب داخل استعلام آخر بين قوسين (...) يُنفَّذ أولاً، وتُستخدم نتيجته داخل الاستعلام الخارجي.

رأينا سابقاً استعلام يستخدم Having:

```
SELECT department, AVG(salary) AS avg_salary
FROM Employees
GROUP BY department
HAVING AVG(salary) > 4000;
```

يمكن تحقيق نفس النتيجة باستخدام الـ subquery :

Subquery

يعيد جدول
(علاقة) يحوي
متوسط الرواتب

```
SELECT *
FROM (
    SELECT department, AVG(salary) AS avg_salary
    FROM Employees
    GROUP BY department
) AS dept_avg
WHERE avg_salary > 4000;
```

لاحظ أننا استخدمنا WHERE خارج الاستعلام الفرعي.
وذلك لأن:

عندما نضع استعلاماً يحتوي على GROUP BY و AVG() أو أي دالة تجميع أخرى داخل قوسين ونعامله كـ subquery داخل جملة FROM، فإن هذا الاستعلام يُنفَّذ أولاً ويُعاد كـ جدول مؤقت يحتوي على نتائج مجمعة. وبالتالي، يمكننا استخدام WHERE لتصفية هذه النتائج كما لو أننا نتعامل مع جدول عادي. أي أنه في هذه الحالة يمكن اعتبار :

خارجي WHERE + subquery داخل GROUP BY

====GROUP BY + HAVING



حالات استخدام لل SUBQUERY

1. SELECT * FROM (subquery) :

تفيد في تجزئة الاستعلامات المعقدة أو لتجهيز جدول مؤقت داخل الاستعلام.

```
SELECT *
FROM (
    SELECT department, COUNT(*) AS emp_count
    FROM Employees
    GROUP BY department
) AS dept_summary;
```

وضعنا نتيجة الاستعلام الجزئي في dept_summary مما يمكننا من استخدامها في استعلام اخر.

2. IN و NOT IN مع Subquery :

IN (subquery)

يتحقق مما إذا كانت قيمة العمود موجودة ضمن قائمة قيم يُعيدها الاستعلام الفرعي.

NOT IN (subquery)

يتحقق أن القيمة ليست موجودة ضمن النتائج.

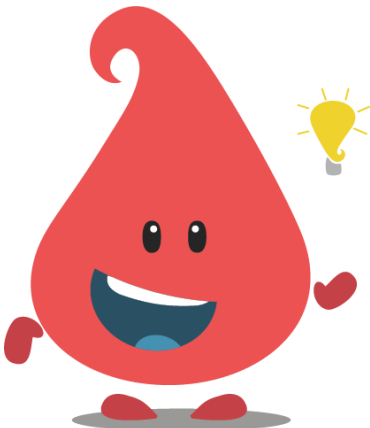
⚡ شروط استخدام IN :

- يجب أن يُعيد الـ subquery عمودًا واحدًا فقط (single column)
- نوع البيانات في العمود الخارجي يجب أن يتوافق مع نوع العمود في نتيجة الـ subquery

مثال:

```
SELECT id FROM (subquery that returns an ids column)
```

- تُستخدم في المواضع التي تتطلب قيمة واحدة في كل صف تتم مقارنتها بمجموعة.



الموظفون الذين يعملون في أقسام فيها أكثر من موظفين اثنين:

```
SELECT *
FROM Employees
WHERE department IN (
    SELECT department
    FROM Employees
    GROUP BY department
    HAVING COUNT(*) > 2
);
```

الموظفون الذين لا يعملون في هذه الأقسام:

```
SELECT *
FROM Employees
WHERE department NOT IN (
    SELECT department
    FROM Employees
    GROUP BY department
    HAVING COUNT(*) > 2
);
```

3. SOME و ALL مع Subquery :

Some: تُستخدم مع مقارنة شرطية مثل (<, >, =, <=, >=, ><)

للتحقق مما إذا كانت القيمة في الاستعلام الخارجي تُطابق أحد القيم في نتيجة

ال Subquery

ترجع TRUE إذا كان الشرط ينطبق على أي صف واحد على الأقل من نتيجة الـ subquery.

أهم شرط عند استخدامها أن الـ subquery يجب أن يُعيد قائمة من القيم (عمود واحد فقط).

ALL: تعمل بعكس SOME

ترجع TRUE إذا كان الشرط ينطبق على جميع الصفوف في نتيجة الـ subquery.

أمثلة:

- الموظفون الذين راتبهم أكبر من بعض (واحد على الأقل) الموظفين في قسم HR:

```
SELECT *
FROM Employees
WHERE salary > SOME (
    SELECT salary FROM Employees WHERE department = 'HR'
);
```

يعرض الموظفين الذين راتبهم أكبر من أحد رواتب موظفين ال HR على الأقل.

- الموظفون الذين راتبهم أعلى من كل رواتب HR:

```
SELECT *
FROM Employees
WHERE salary > ALL (
    SELECT salary FROM Employees WHERE department = 'HR'
);
```

يعرض الموظفين الأعلى راتباً من كل من في HR.

ملاحظة أربيسيزية:

إذا كانت نتيجة ال subquery تحتوي على NULL، فإن المقارنة بـ ALL قد تُعيد UNKNOWN يجب أن يُعيد ال subquery قائمة صالحة للمقارنة.

4. EXISTS و NOT EXISTS مع Subquery :

- EXISTS (subquery) : يتحقق مما إذا كان الاستعلام الفرعي يُعيد صفوفاً (واحدة أو أكثر)
- NOT EXISTS (subquery) : يتحقق مما إذا كان الاستعلام الفرعي لا يُعيد أي صف.

الفرق الجوهرى عن IN:

EXISTS لا تهتم بما تُعيده الأعمدة، بل فقط هل هناك أي صف ناتج أم لا.

قوة EXISTS:

يمكنها الوصول إلى الجدول الخارجي داخل subquery بعكس IN.

```

SELECT * FROM Employees e
WHERE EXISTS (
  SELECT * FROM Projects p
  WHERE p.emp_id = e.emp_id
);

```

استعلام يعرض الموظفين الذين لديهم مشاريع (ال ID الخاص بهم له وجود في جدول المشاريع).
 نلاحظ أن الـ subquery يستخدم العمود من الجدول الخارجي e.emp_id.

أمثلة أخرى:

■ الموظفون الذين يعملون في قسم يحتوي على موظفين لهم راتب فوق 5000:

```

SELECT *
FROM Employees e
WHERE EXISTS (
  SELECT * FROM Employees
  WHERE department = e.department AND salary > 5000
);

```

"أعرض كل الموظفين من الجدول، بشرط أن يكون القسم الذي يعمل فيه الموظف يحتوي على موظف واحد على الأقل راتبه أكبر من 5000".

■ الموظفون في أقسام لا تحتوي على أي موظف راتبه فوق 5000:

```

SELECT *
FROM Employees e
WHERE NOT EXISTS (
  SELECT * FROM Employees
  WHERE department = e.department AND salary > 5000
);

```

5. UNIQUE(subquery) :

تتحقق من أن نتيجة الاستعلام الفرعي لا تحتوي على صفوف مكررة.

تعيد TRUE إذا كانت كل الصفوف الناتجة فريدة، أو لم تعد أي صف.

■ مثال:

```
SELECT * FROM Employees
WHERE UNIQUE (
  SELECT salary FROM Employees
  WHERE
  department = 'HR' AND salary = Employees.salary
);
```

■ "اعرض الموظف فقط إذا كان راتبه يظهر مرة واحدة فقط في HR."

ملاحظة اربيسيزية:

■ UNIQUE نادرة الاستخدام، وغير مدعومة حرفياً في SQL Server، و يمكن محاكاة المثال السابق باستخدام:

```
SELECT * FROM Employees
WHERE salary IN (
  SELECT salary
  FROM Employees
  WHERE department = 'HR'
  GROUP BY salary
  HAVING COUNT(*) = 1
);
```

■ هذا يتحقق أن الراتب يظهر مرة واحدة فقط في HR - نفس تأثير UNIQUE.

6. استخدام subquery داخل أعمدة SELECT:

مثال: بديل ال group by

بدلاً من استخدام :

```
SELECT company_id, COUNT(*) FROM Employees GROUP BY company_id;
```

لجلب عدد الموظفين في كل شركة نستطيع استخدام subquery داخل SELECT لكل صف من جدول Companies:

```
SELECT
  c.name,
  (
    SELECT COUNT(*)
    FROM Employees e
    WHERE e.cid = c.id
  ) AS employee_count
FROM Companies c;
```

الشرح:

- لكل شركة c ، نقوم بتشغيل subquery يقوم بعدد الموظفين في جدول Employees الذين ينتمون إليها عن طريق c.id

- ناتج الاستعلام يحتوي على اسم الشركة وعدد موظفيها، دون استخدام GROUP BY.

ملاحظة: يجب أن يكون ناتج ال subquery هو سطر واحد حصراً و الا يحدث خطأ.

7. الاستعلامات الفرعية (Subqueries) داخل أوامر INSERT, UPDATE و DELETE:

بمعنى آخر في ال (DATABASE MODIFICATION):

1. INSERT INTO ... (SELECT ...)

إدخال بيانات من جدول إلى جدول آخر

```
INSERT INTO Employees (name, address)
```

```
SELECT name, address
```

```
FROM Company;
```

هذا الاستعلام يقوم بـ:

جلب name و address من جدول Company وإدخالهم في جدول Employees بنفس الترتيب.

شرط مهم: يجب أن يتطابق عدد الأعمدة وأنواعها في الجدولين.

2. UPDATE باستخدام Subquery

تحديث قيمة في جدول بناءً على نتيجة من جدول آخر

UPDATE Employees

SET name = (

SELECT c.name

FROM Company c

WHERE Employees.id = c.id

);

هذا الاستعلام يقوم بـ:

تحديث حقل name في جدول Employees ويأخذ الاسم من جدول Company حيث id متطابق.

شرط مهم:

- يجب أن يُعيد subquery صفًا واحدًا فقط لكل موظف وإلا سيحدث خطأ

subquery returned more than one row

3. DELETE باستخدام Subquery

حذف صفوف بناءً على شرط مستخرج من جدول آخر

DELETE FROM Employees

WHERE id IN (

SELECT id FROM Company WHERE status = 'closed'

);

هذا الاستعلام يقوم بـ:

حذف الموظفين الذين توجد أرقامهم (id) في جدول Company بشرط أن status = 'closed' هذه الطريقة مفيدة لحذف صفوف مرتبطة بشرط في جدول خارجي.

رابعاً التعامل مع الشروط : CASE – END

الفكرة العامة

جملة CASE تُستخدم في SQL لتطبيق شرط منطقي داخل SELECT (أو داخل WHERE, ORDER BY وغيرها)، وتشبه إلى حد كبير if ... else في لغات البرمجة.

الصيغة العامة:

CASE

When الشرط_الاول then القيمة_الاولى

When الشرط_الثاني then القيمة_الثانية

.....

Else القيمة_ الافتراضية

End

مثال عملي:

وصف العمر

نفترض أن لدينا جدول Students يحتوي على عمود age ، ونريد أن نعرض وصفاً للعمر:

أقل من 18 → "قاصر"

بين 18 و 25 → "شاب"

أكثر من 25 → "بالغ"

أي قيمة غير معروفة → "غير محدد"

الاستعلام:

SELECT

name,

age,

CASE

WHEN age < 18 THEN 'قاصر'

WHEN age BETWEEN 18 AND 25 THEN 'شاب'

WHEN age > 25 THEN 'بالغ'

ELSE 'غير محدد'

END AS age_description

FROM Students;

ملاحظات أريسيزية:

- يمكنك استخدام CASE في أي مكان داخل SQL تقريباً:
- في SELECT لإظهار بيانات مشروطة.
- في ORDER BY لترتيب حسب حالات.
- في WHERE لحالات أكثر تعقيداً (معقدة قليلاً).
- يمكن استخدام ELSE أو تركها فارغة (لكن الأفضل دائماً إضافتها لتغطية الحالات غير المتوقعة).
- CASE تتوقف عند أول شرط يتحقق (short-circuit logic).

